A-Graph: A Unified Graph Representation for Cross-Stack Cost Modeling

Daniel Price[1], Prabhu Vellaisamy[2], Patricia Gonzalez[3], George Michelogiannakis[3], John P. Shen[2], Di Wu[1]
[1]Department of ECE, University of Central Florida, Orlando, FL, USA
[2]Department of ECE, Carnegie Mellon University, Pittsburgh, PA, USA
[3]Lawrence Berkeley National Laboratory, Berkeley, CA, USA
daniel.price@ucf.edu, pvellais@andrew.cmu.edu, lg4er@lbl.gov, mihelog@lbl.gov, jpshen@cmu.edu, di.wu@ucf.edu

arXiv:2602.04847v3 [cs.PF] 27 Aug 2026

***Abstract*—Emerging systems, such as superconducting and neuromorphic, are driving a new era of computing with increasingly diverging design requirements compared to conventional systems. These rapidly evolving design spaces require agile development and early design-space exploration (DSE) to identify promising performance-cost tradeoffs. While conventional systems benefit from mature toolchains, emerging systems lack comparable support. Existing cost estimators target specific stacks, while hardware-generation frameworks presume mature compilers and EDA flows absent in such emerging technologies.**

To fill this gap, we introduce Architecture-Graph (*A-Graph*), a graph that unifies the cross-stack system representation, realized through *Archx* for exploring the large architecture design space. *A-Graph* and *Archx* form a cross-stack *cost-modeling* framework that separates cost aggregation from performance models, enabling tightly coupled cost co-simulation.

***A-Graph* and *Archx* facilitate *usability* in three key aspects: ❶ *A-Graph* supports cross-stack system representation and metric definition, enabling *flexibility*. ❷ *Archx* automatically generates and sweeps design points under user constraints, improving *programmability*. ❸ *Archx* adopts scope-based metric retrieval and aggregation exposure, reporting cost at any user-defined hierarchy, enhancing *explainability*.**

Case studies spanning emerging systems (superconducting, neuromorphic) and conventional architectures (RISC-V, GPU) demonstrate high fidelity and showcase usability. Overall, *A-Graph* and *Archx* unlock cross-stack cost modeling for rapid and accurate DSE.

I. Introduction

Computer architecture is entering a golden age [1], with new computing systems flourishing. Beyond traditional CMOS, more exotic circuit technologies are emerging, such as memristors [2]–[7], superconducting [8]–[14], photonics [15]–[20], quantum [21]–[26], and more (even mushroom [27]). Each technology instigates new architectural organizations: memristors ignite neuromorphic computing [28]–[31], while superconducting and photonic devices motivate new architectures for artificial intelligence (AI) [11], [32], [33] and quantum computing [9], [34], [35]. New applications naturally arise, such as AI for protein structure prediction [36], physics simulation [37], cryogenic sensing [38], [39], and more.

Within these new systems, a wide range of simulators exist. Circuit simulators support emerging paradigms such as memristors [48], superconducting [49], photonics [50], etc. Architecture simulators explore computer architecture and memory design [40], [41], [51]–[55]. Application simulators focus on specific workloads, such as AI with general matrix multiplication (GEMM) [47], [56]–[58], visual computing [59], unary computing [60]–[63], neuromorphic computing [64], [65], robotics [66], etc. Cost simulators, e.g., Accelergy [46], target varying stacks for metric cost analysis. Hardware-generation frameworks generate hardware directly from user specifications or programs [43]–[45].

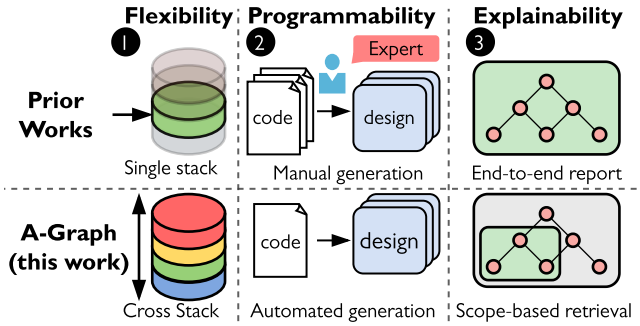

Fig. 1: Usability challenges.

While these new systems promise substantial gains, evaluating their performance and cost within existing frameworks remains challenging. Their design spaces are vast and rapidly evolving, lowering the efficacy of these specialized simulators in the near future and imposing *usability challenges*.

Usability challenges: Figure 1 outlines three usability challenges we have identified where existing frameworks fall short.

- ***Flexibility***: While few existing frameworks lack cross-stack support, most are specialized for a target stack. Moreover, they are often hardwired to specific metrics, limiting cross-stack co-design and hindering *flexibility* as emerging systems arise.
- ***Programmability***: DSE in existing frameworks often requires manually defining configurations. This can be time-consuming when exploring thousands of design points, and error-prone as singular mistakes can invalidate comparative results. Moreover, some existing works require high cross-stack expertise when designing in their frameworks. This lack of programmability hinders the rapid design process that emerging systems demand.
- ***Explainability***: Existing frameworks offer high fidelity, but often limit reported metrics to end-to-end results.

TABLE I: Comparing simulators for hardware cost estimation. Green cells indicate the best usability in each category.

Design	Goal		Flexibility			Programmability		Explainability	
			Stack	Metric	Target	Expertise	Sweeping	Scope	Aggregation
Charm [40]	Pre-RTL	Perf / cost	Arch	*User-defined*	Analytical	*Arch behavior*	*Automated*	End-to-end	Obscured
Aladdin [41]	Pre-RTL	Perf / cost	*Cross-stack*	PPA	CMOS	C	Partially auto	Component	Obscured
SNSv2 [42]	Post-RTL	Perf / cost	Arch/Circuit	PPA	CMOS	RTL	Manual	End-to-end	Obscured
DSAGEN [43]	Post-RTL	Chip gen	*Cross-stack*	PPA	CMOS Acc	C & #pragma	Partially auto	End-to-end	Obscured
OverGen [44]	Post-RTL	Chip gen	*Cross-stack*	PPA	CMOS FPGAs	C & #pragma	Partially auto	End-to-end	Obscured
TNNGen [45]	Post-RTL	Chip gen	*Cross-stack*	PPA	CMOS TNN	C & #pragma	Manual	End-to-end	Obscured
Accelergy [46]	Pre-RTL	Cost	*Cross-stack*	Energy	Timeloop [47]	*Arch behavior*	Manual	Component	Obscured
Archx (ours)	Pre-RTL	Cost	*Cross-stack*	*User-defined*	*User-defined*	*Arch behavior*	*Automated*	*Scoped*	*Exposed*

PPA: Performance, power and area. TNN: Temporal neural network (Neuromorphic).

Additionally, those with hierarchical reporting do not expose how these metrics are computed. Without explainability, understanding interactions in emerging systems with unknown design spaces is difficult.

Table I compares prior works along these axes of usability. Each simulator is effective within its target stack(s), where specialized cost models and compilation frameworks lend themselves to high fidelity. However, this specialization causes each to fall short in at least one aspect of usability.

By targeting specific stacks or metrics, these frameworks lack the flexibility to adapt to emerging systems. Extending them would not only require modifying their performance models, but also redesigning their underlying cost models or compilation frameworks. Even with modifications, existing frameworks would remain limited for DSE, as they lack the programmability and explainability required to efficiently explore and interpret vast design spaces. Moreover, continuously redesigning frameworks as new systems emerge delays development, consuming effort that could otherwise be spent exploring and optimizing design decisions. These deficiencies raise a key question: *Can a single framework provide the flexibility, programmability, and explainability needed to enable cross-stack cost analysis and DSE for emerging systems?*

Proposal: In response to this challenge, we introduce Architecture-Graph (*A-Graph*), a graph that unifies the representations of system stacks. Unlike prior works that are constrained to target stacks or predefined metrics, *A-Graph* allows designers to customize their own performance models and cost metrics. Represented in the same graph, performance models break down the full system into hierarchical events that ultimately link to their own cost. This representation lets designers model target emerging systems without hard-wiring metric aggregation. Moreover, designers can now select the simulation fidelity by providing the right performance models and cost metrics at the target granularity, where existing hardwired frameworks fail.

To unlock *A-Graph*'s full potential, we develop *Archx*, a cost-modeling framework that implements *A-Graph*. *Archx* consists of a front-end programming interface for configuration sweeping, which automatically generates and sweeps only valid design points in the vast design space. Additionally, *Archx* provides scope-based metric retrieval and aggregation exposure, allowing designers to both evaluate system metrics at scopes of interest and reveal how such metrics are computed. This programmability equips *A-Graph* for the rapid DSE emerging systems require, while the explainability exposes system interactions within *A-Graph*.

Together, *A-Graph* and *Archx* form a cost-modeling framework that separates cost aggregation from performance models that drive it. With this separation, representing a system only requires supplying its performance models and cost metrics, while cost aggregation stays fixed. This yields tightly coupled cost co-simulation without rebuilding the framework, addressing the usability challenges above.

Here we outline how we position *A-Graph* in the ecosystem.

What *A-Graph*/*Archx* is

- a unified representation for cross-stack cost evaluation.
- a decoupled framework of cost aggregation from user-supplied performance models and metrics.
- an automated approach for cost modeling towards DSE.
- a framework for scope-based metric retrieval and aggregation exposure.

What *A-Graph*/*Archx* is not

- a simulator with built-in performance models or metrics.
- a framework restricted to a specific stack or technology.
- an EDA or compiler toolchain.

The contributions of this work are as follows:

- We introduce *A-Graph*, a unified cross-stack graph representation that separates cost aggregation from user-supplied performance models and customizable metrics, enabling flexible cost evaluation for emerging systems.
- We develop *Archx*, a cost-modeling framework realizing *A-Graph*, that automatically sweeps design points under user constraints, improving programmability, and exposes cost at user-defined scopes, enhancing explainability.
- We validate *A-Graph* and *Archx* through case studies covering both emerging and conventional systems, across different technologies, architectures, and applications, achieving high fidelity.

This paper is organized as follows. Section II motivates *A-Graph*. Sections III and IV detail *A-Graph* and *Archx*. Sections V and VI present case studies. Sections VII and VIII discuss and conclude this paper.

II. Motivating Usability

Section I named three usability challenges that prevent existing frameworks from serving emerging systems: flexibility, programmability, and explainability, detailed in Table I.

A. Flexibility

Most existing frameworks are constrained to their target stack(s), limiting their flexibility to emerging systems. Some of these frameworks are able to cross the stack within such constraints. Accelergy can attribute cost only for AI tensor workloads by integrating with Timeloop [46], [47]. Aladdin estimates cost directly from C [41], while DSAGEN and OverGen generate RTL from C and compiler directives [43], [44]. All three are confined to CMOS, while the latter two are further confined to spatial accelerators or FPGAs. TNNGen is limited to neuromorphic workloads, generating RTL from PyTorch implementations [45]. Other works focus on one specific stack. Charm only models architecture interactions [40], while SNSv2 estimates synthesis results from RTL [42].

Additionally, such frameworks hardwire metrics that are needed for their target stack(s). Accelergy reports only energy, while most other works are limited to CMOS metrics, reporting only performance, power, and area (PPA). A cost outside this built-in set cannot be expressed at all. For example, area for superconducting logic is measured in Josephson junctions (JJ) rather than mm^2 for CMOS, which no existing framework can be extended to support. Charm alone allows user-defined metrics, though only at an analytical level.

Mature EDA tools deliver precise PPA estimates, but each is similarly bound to a single technology and stack (e.g., CMOS at the circuit level) and to metrics that technology supports. Emerging technologies such as superconducting and memristor logic lack such mature tools, though recent works begin to address this [67]–[70].

Existing frameworks lack the flexibility that emerging systems require, motivating a framework that represents cross-stack system interactions with customizable metrics.

B. Programmability

Effective DSE requires rapidly generating and evaluating many configurations, which is time-consuming in existing frameworks. Without constraining the design space to only valid configurations, time is wasted evaluating invalid designs. Assume a designer wants to explore general matrix multiplication (GEMM) on a systolic array. This involves sweeping across the array size, bitwidth, dataflow, buffer size, GEMM dimensions (M, K, N), tiling, etc. With only two options per knob, these eight knobs already give 256 combinations, a count that grows exponentially with every knob or configuration added. Conversely, frameworks that do not include automation under constraints require the designer to manually write configurations, which is error-prone. If a single mistake is made among thousands of combinations, then the comparative results are invalidated, and thus the conclusion.

Existing frameworks include varying levels of automation, but none fully automate the cross-stack DSE. Accelergy [46], SNSv2 [42], and TNNGen [45] offer no automation at all. Aladdin [71], DSAGEN [43], and OverGen [44] automate part of the space, holding the software fixed and sweeping only the hardware. Exploring alternative software thus means re-implementing it for each design point. Charm [40] alone includes full automation, but only over analytical architecture models.

Beyond automated DSE, most frameworks demand high cross-stack expertise. Aladdin requires the designer to be proficient in parallel C programming [41]. A designer without extensive parallel-programming knowledge will make naive design choices, such as generating non-vectorized hardware. Similarly, DSAGEN [43] and OverGen [44] require the designer to express parallelism in C with compiler directives, and SNSv2 [42] requires RTL expertise.

No single framework supplies the programmability to automate cross-stack DSE, motivating a framework that automates the exploration of emerging systems.

C. Explainability

Understanding how a design decision drives performance and cost requires visibility of finer-grain scope within the system. When systems are reduced to a single end-to-end result, designers are left guessing the hierarchical behavior. Using the same GEMM example from Section II-B, mapping tiles to the array, and the computation itself, carry separate costs that aggregate into the end-to-end total. To isolate one of these events, a designer must define the scope of the metric and report it at that granularity, which existing frameworks do not allow. Moreover, if frameworks do not expose how these metrics are aggregated, designers are left unable to trace a cost back to the interactions that produced it.

Most existing frameworks report only a single end-to-end number, obscuring fine-grain system behavior. Charm [40], SNSv2 [42], DSAGEN [43], OverGen [44], and TNNGen [45] report only a single end-to-end result. Accelergy [46] and Aladdin [41] report at a finer per-component scope, yet neither lets the user define it nor reveals how each number is aggregated. EDA tools give hierarchical PPA, but that hierarchy stops at the circuit level. Recent EDA works have begun to pursue this explainability [72]–[75], but none offers a comprehensive cross-stack view.

This lack of explainability leaves designers of emerging systems unable to attribute cost, motivating a framework that exposes these fine-grained interactions.

III. *A-Graph* Representation

To address flexibility, we introduce Architecture-Graph (*A-Graph*), a unified representation that captures interactions across the system stack. Figure 2 illustrates *A-Graph*, representing a user-defined system across the *application*, *software*, *architecture*, and *circuit* stacks. Moreover, *A-Graph* separates cost aggregation from performance models, enabling tightly coupled cost co-simulation. Finally, *A-Graph* is designed to accept user-defined metrics that are relevant to the system being represented. Through these, *A-Graph* is flexible enough to extend to new systems as they emerge.

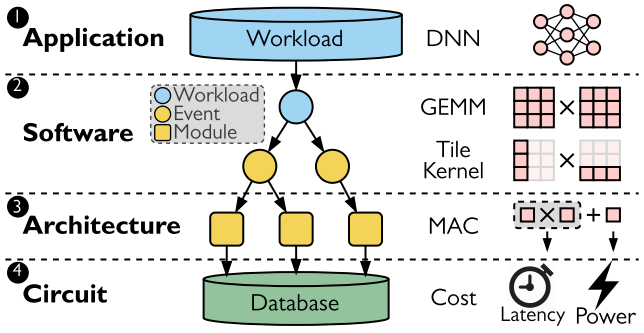

Fig. 2: Overview of *A-Graph*.

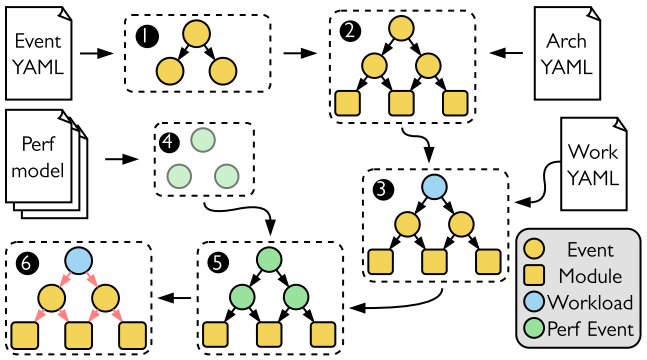

Fig. 3: *A-Graph* construction.

A. Unified Representation

A-Graph represents the full system within a single event-based graph, unifying the application, software, architecture, and circuit stacks. Each stack is expressed through events and their decomposition, i.e., how they break down into sub-events, capturing interactions across layers within the same graph. Representing the stack and its interactions hierarchically, *A-Graph* lets designers fully describe the target system without being constrained to a particular stack. This allows *A-Graph* to separate the system representation from the cost modeling that drives it, resulting in a tightly coupled cost co-simulation.

In Figure 2, *A-Graph* is organized into four system stacks: *application*, *software*, *architecture*, and *circuit*.

❶ **Application:** At the top of the stack, the application layer represents a user-defined specification of the high-level program or algorithm that serves as the input workload to *A-Graph*, highlighted as the *blue* root event. By leaving the application layer user-defined, *A-Graph* extends easily to emerging applications without requiring a fixed compiler flow or pre-existing software stack.

As an example (Figure 2), consider a deep neural network (DNN) model as the application. This DNN workload has parameters (e.g., batch size, layer count, hidden dimension) that describe the workload and drive the decomposition of the application into software events.

❷ **Software:** The software layer builds out the middle of the graph, decomposing the workload into architecture events and driving the hierarchical mapping within *A-Graph*. For each software event, designers provide performance models that generate weighted edges to sub-events, quantitatively encoding this decomposition. Each performance model is isolated to the specific software event it is designed for. This runs performance models locally, effectively separating them from cost aggregation. Moreover, the event granularity is user-defined, supporting the trade-off between evaluation speed and fidelity.

Building on the DNN example, the performance models decompose the DNN into GEMM layers, then further into tile sizes that match the compute array.

❸ **Architecture:** The architecture layer defines the micro-architecture blocks that compose the system design. These architecture modules (e.g., compute units, memory blocks, gates) are instantiated as sub-events of the software layer's leaf nodes. The architecture layer is a flattened description that makes no assumptions about structural hierarchy, instance relationships, or spatial dependencies. This abstraction keeps module definitions fully user-configurable, letting designers adjust granularity and representation to their modeling needs.

Continuing the DNN example, the matrix tiles from the software layer map to the compute array and break down into the multiply-accumulate (MAC) operations and other hardware events (e.g., memory, registers, FIFOs) each tile requires.

❹ **Circuit:** At the bottom of the stack, the circuit layer connects architecture modules to technology-specific metrics. Each module references a database of precomputed circuit-level results (e.g., from EDA tools), or outputs from cost simulators (e.g., CACTI7 [54]), enabling rapid retrieval of accurate metrics without performing full system simulation. By allowing users to define where metrics are sourced, e.g., CMOS and mm^2, FPGAs and LUTs, superconducting and JJs, *A-Graph* can easily extend to emerging systems without requiring mature EDA tools or pre-existing cost models.

Each architecture module (e.g., a MAC unit) carries a hardware cost (e.g., area, power) that supplies *A-Graph* with circuit-level metrics for aggregation. This decomposition produces a graph that represents a unified system across all stacks.

B. Graph Construction.

Figure 3 depicts how *A-Graph* is constructed. *A-Graph* starts as a directed acyclic graph (DAG), where nodes represent system events and edges point from events to their sub-events. Each non-leaf event is supplied with a performance model that defines how it decomposes into sub-events.

Besides the circuit stack, which is either represented by a database of precomputed costs or retrieved from a plugged-in simulator, each stack is defined with a YAML file. In Figure 3, ❶ the graph starts with the event YAML, or software stack, which defines the graph structure of the system. ❷-❸ add the application workload to the root and architecture modules to the leaves of the graph. ❹-❺ map the performance models, which detail how events decompose into sub-events, onto the event graph. Finally, ❻ applies the performance models, supplying each edge with a weight.

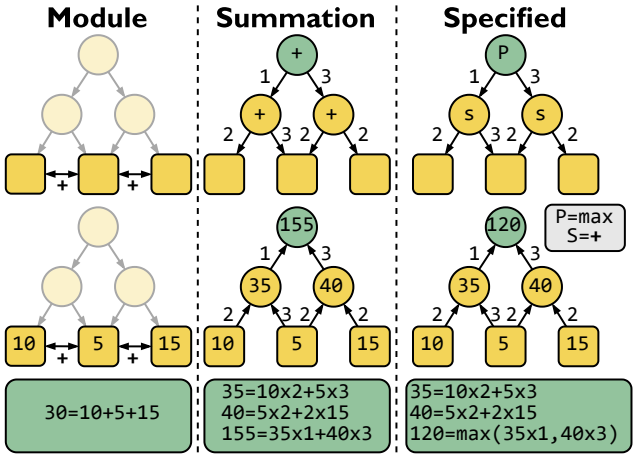

Fig. 4: *A-Graph* metric aggregation. Circles represent events, while squares represent modules. The top graph illustrates each aggregation functionality, while the bottom graph provides the aggregation flow and final value, shown in green. The green boxes display equivalent equations.

This transforms *A-Graph* from a DAG to a weighted directed acyclic graph (WDAG). The resulting WDAG captures the user-defined behavior across every stack and is ready to report cost across the system.

C. Metric Aggregation

A-Graph supports customized metric aggregation, letting designers define metrics for the target system and how those metrics aggregate into cost. This customization allows *A-Graph* to extend to emerging systems whose cost is defined by unique metrics.

Metric aggregation proceeds by traversing the graph in topological order, aggregating along weighted edges from module nodes up the graph. Weighted edges are treated multiplicatively, scaling metrics according to the number of times a child event is invoked by its parent. This process produces system-level cost by aggregating values from user-defined metrics (e.g., area, power, latency), exposing how the system's cost breaks down.

To give designers cost beyond end-to-end results, *A-Graph* provides scope-based metric retrieval. Designers select the scope at which metric retrieval occurs, aggregating only a subgraph within the system graph. This subgraph can be constructed by selecting any application or software event as its scope. As an example, let us revisit the DNN system from Figure 2. If a designer wanted to find the cost of a tile mapping instead of the entire DNN workload, then a tile event could be selected as the scope. The resulting cost would only consider this tile event and those below it, giving insight into costs that build the end-to-end results.

A-Graph currently supports three types of aggregation, visualized in Figure 4.

- ***Module***: Directly aggregates metrics of architecture modules, independent of application and software influences (e.g., area, leakage power).
- ***Summation***: Accumulates metrics across edges, collapsing to a single value at the root node. This applies to aggregation with no temporal dependencies, such as dynamic energy.
- ***Specified***: Supports two interaction modes, *sequential* and *parallel*. Sequential events, similar to summation, accumulate incoming metrics, while parallel events take the maximum, modeling interactions like runtime where parallel execution is bounded by the slowest event. Dependency-induced stalls and pipelining are not captured by aggregation and must come from the user's performance model or an external trace.

A-Graph's metric aggregation is as flexible as a spreadsheet [76]–[78], but tailored to cross-stack cost modeling for user-defined systems. Therefore, we coin it as *a topological spreadsheet*. By aggregating cost at user-defined scopes, *A-Graph* is flexible and explainable.

IV. *Archx* Framework

To further address programmability and explainability, we develop *Archx* to implement *A-Graph* at its core for fast DSE, with the workflow shown in Figure 5. In Phase 1, *Archx* provides a user-friendly programming interface to automatically generate and sweep design points under user-defined constraints. Then, phase 2 constructs an *A-Graph* for each configuration. Finally, Phase 3 incorporates scope-based metric retrieval and aggregation exposure to analyze and understand each design point at user-defined granularity.

A. Phase 1: Front-end Sweeping

Archx supports a user-friendly front-end interface to define and constrain systems. The system description is then automatically swept across valid design points within the provided constraints, generating description files. Moreover, *Archx* does not require developers to have cross-stack expertise. For example, Aladdin [41] mandates optimal C code for processor design. The following sections detail the process of defining, constraining, and generating the design space in *Archx*.

Listing 1 presents the programming interface with a similar example as Figure 2, using a MAC array (MA) running a tiled GEMM workload. At the start of the listing (lines 1-3), an *A-Graph* object and its descriptions are instantiated.

Workload Description: The workload description (lines 5-9) defines a workload (i.e., *GEMM*) and its parameters (i.e., batch size (*batch*) and matrix dimension (*dim*)).

Archx provides two methods for defining workload parameters: static and sweeping. Static parameters use a value as-is. Here, the batch is static, only considering a single batch size. Sweeping parameters iterate over the provided list. The dim parameter is a sweeping parameter, which will generate configurations for each element within the list. To distinguish a sweeping parameter from a static one, an additional sweep argument is needed, as seen on line 9.

Event Description: Each workload must then be defined as a root node in the software description (lines 11-13), creating an

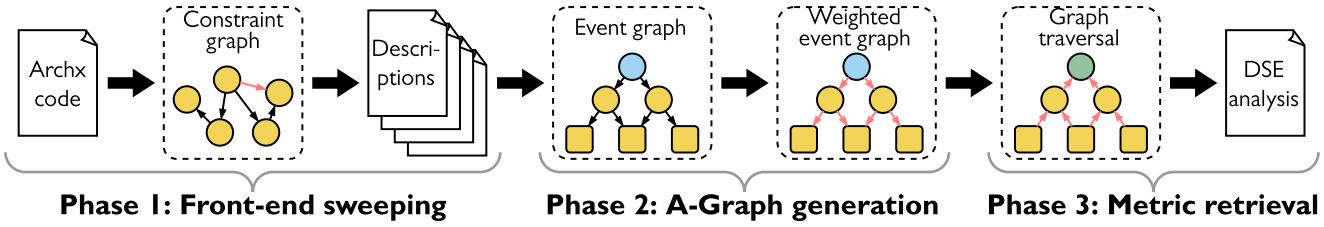

Fig. 5: Overview of *Archx*.

```
1  def description(path):
2    agraph = AGraph(path=path)
3    arch, event, mtrc, wkld = agraph.desc
4
5    # Add workloads (Application)
6    W = wkld.add(['GEMM'])
7    batch = W.add(name='batch', value=4)
8    dim = W.add(name='dim', value=[32, 64],
9                sweep=True)
10
11   # Add events (Software)
12   event.add(name='GEMM', subevent=['tile'])
13   event.add('tile', ['sram', 'mac', 'p-reg'])
14
15   # Add modules and attributes (Architecture)
16   arch.add_attr(tech='7nm', interface='cmos')
17   arr_inst = [[x*2, x*2] for x in range(2, 6)]
18   sram_width = [[x*2] for x in range(2, 6)]
19
20   mac=arch.add(name='mac', inst=arr_inst,
21     tag=['pe'], query={'class':'mac'})
22   pReg=arch.add(name='p-reg', inst=arr_inst,
23     tag=['pe'], query={'class':'reg', 'width': 16})
24   sram=arch.add(name='sram', inst=[1], tag=['mem'],
25     query={'class':'sram', 'interface':'cacti7',
26            'depth':128, 'width':sram_width})
27
28   # Add metrics (Circuit)
29   mtrc.add(name='area', unit='mm^2', aggr='module')
30   mtrc.add(name='energy', unit='nJ', aggr='sum')
31   mtrc.add(name='runtime', unit='ms', aggr='spec')
32
33   # Add sweeping constraints (Constraint)
34   agraph.injection([pReg['inst'], mac['inst']])
35   agraph.exclusion(a=mac['inst'], b=dim,
36                    cond=lambda a, b: a[0] == b)
37   agraph.condition(a=mac['inst'], b=sram['width'],
38                    cond=lambda a, b: a[0] == b)
39   return agraph.generate()
```

Listing 1: MAC array front-end code.

event with the same name as the workload. Events propagate from root to leaf events by recursively invoking sub-events until reaching the architecture modules. The root *GEMM* then defines *tile* as its sub-event, which further decomposes into the architecture modules *mac*, *p-reg*, and *sram*. Each event is associated with a performance model that specifies such decomposition and computes the edge weights to each sub-event, e.g., how many *tiles* under *GEMM*.

Architecture Description: The architecture description instantiates modules and defines their attributes (lines 15-26). First, global attributes are default for all architecture modules, and can be overwritten by local attributes within each module. This is highlighted in the *sram* module (line 24), which uses the *cacti7* interface as its cost database instead of the global *cmos* interface for compute logic.

Then, each module is instantiated with three key properties: *instance*, *tag*, and *query*. The *instance* property specifies the number of module instances, which can also be static (a list) or sweeping (a list of lists), similar to workload parameters. Next, the *tag* property groups modules for scoped metric retrieval (Section IV-C). Finally, the *query* property defines the configuration to retrieve metrics from a module database, specified by class. Each query is user-defined by the database, consistent with *A-Graph*'s flexibility for defining new metrics and cost databases.

Metric Description: The metric description lists the available metrics and specifies how they are aggregated when traversing the graph for metric retrieval. Each metric is defined with a name, unit, and aggregation type, e.g., *area*, *energy*, and *runtime* (Section III-C).

Constraint Graph: By default, *Archx* sweeps all parameters, which generates invalid design points. To constrain the design space to valid parameter combinations, *Archx* provides design constraining to remove invalid or unwanted configurations. This sweeping is implemented with three types of constraints: *Injection*, *Exclusion*, and *Condition*.

- *Injection* constraints enforce a one-to-one mapping between parameters, such as aligning the MA array's processing elements and pipeline registers.
- *Exclusion* constraints remove invalid parameter combinations, such as removing MA array sizes that are larger than the matrix dimensions.
- *Condition* constraints enforce a mapping between parameters that meet the condition, while removing combinations that do not, such as ensuring the MA array size aligns with the SRAM width.

Figure 6 outlines how designs are constrained in *Archx*. It represents the design space as a constraint graph constructed from the system description and user-defined constraints. ❶ Static objects, such as event and metric descriptions, are directly forwarded to generate YAML description files. Sweeping objects, including architecture and workload descriptions, are used to construct the initial constraint graph in ❷. ❸ All *condition* constraints are split into multiple non-condition constraints, to define the valid and invalid combinations from the conditions. ❹ Nodes connected by edges are grouped into subgraphs, and each subgraph is reduced to produce local sweeps. ❺ A global sweep across groups then considers all possible combinations of these subgraphs. ❻ Finally, the swept architecture and workload combinations are joined with static

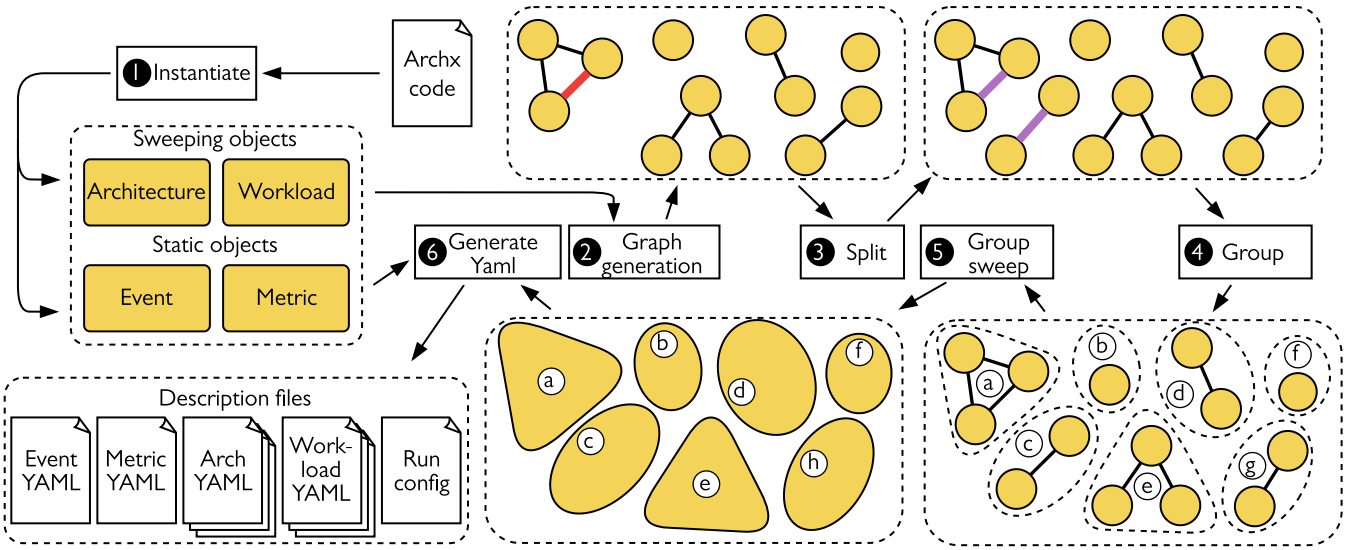

Fig. 6: *Archx* constraint graph and description generation. Nodes represent workload or module parameters, while edges denote constraints. Red edges indicate condition constraints, while purple edges are the result of splitting these conditions.

```
1  def GEMM(arch_dict, wkld_dict):
2    # Retrieve workload and architecture parameters
3    mac_dim = arch_dict['mac']['inst'][0]
4    M, K, N = wkld_dict['config']['dim'] #matrix dims
5    m, k, n = mac_dim[0]     # tile dims
6    cnt = (M/m) * (K/k) * (N/n) # Array mappings
7    tile_dict={'count': cnt, 'aggr':'sequential',
8               'factor':{'runtime':2}}
9    # Add to performance dict
10   perf_dict = {'subevent': {'tile': tile_dict}}
11
12   return perf_dict
13
14 def tile(arch_dict, wkld_dict):
15   # Retrieve architecture parameters
16   perf_dict = {}
17   mac_inst = arch_dict['mac']['inst']
18   reg_inst = arch_dict['p-reg']['inst']
19   bitwidth = arch_dict['p-reg']['query']['width']
20   swidth = arch_dict['sram']['query']['width']
21   freq = arch_dict['mac']['freq']
22
23   # Calculate runtime and sram events
24   sram_events = (mac_inst[0]**3 * bitwidth)/swidth
25   perf_dict['runtime']={'value':1/freq,'unit':'ms'}
26   # Add to performance dict
27   subevents = {{'mac':   {'count': mac_inst}},
28                {'p-reg': {'count': reg_inst}},
29                {'sram':  {'count': sram_events}}}
30
31   perf_dict['subevent'] = subevents
32   return perf_dict
```

Listing 2: Performance model code.

objects to generate the YAML files. This produces description files that sweep the design space, constraining configurations to only valid design points.

Through this front-end interface, *Archx* empowers designers to define *A-Graph* descriptions and constraints, automatically generating and sweeping the design space at all specified design points, demonstrating *Archx* 's high programmability.

B. Phase 2: A-Graph Generation

With the system design constrained and generated, each configuration is then used to construct an *A-Graph*, following the process outlined in Section III-B.

Performance Model: Every event in *Archx* requires a performance model to define the decomposition to its sub-events. Listing 2 details two performance models: the workload root event *GEMM* (lines 1-12) and the leaf event *tile* (lines 13-32).

The *GEMM* model determines the *count* of tile mappings, which is assigned to the edge from *GEMM* to *tile*. Relevant parameters are first retrieved from the architecture and workload descriptions, each represented as a dictionary. Here, the matrix and array dimensions are used to compute the total number of tile mappings to the MAC array. The computed mappings are added to the performance dictionary, then assigned to the event edge. The performance model specifies the *aggregation* for metrics along each edge. Two aggregation types are available, *sequential* and *parallel*, which outline the two options for aggregating specified metrics as detailed in section III-C. Given that our design includes only a single MAC array, one mapping is computed at a time. The runtime of each mapping must wait for the previous to finish, therefore the runtimes accumulate, corresponding to sequential aggregation. With multiple MAC arrays, aggregation could instead be parallel.

Additionally, each edge can have an associated factor, which applies multiplicative scaling of the event count. This is commonly used to parameterize the design or account for hardware utilization. Here, a factor of 2 is applied to runtime to model a non-pipelined MA that requires two cycles.

Unlike the workload model, which focuses on tiling, the leaf event *tile* model specifies computations for its metrics (e.g., runtime in line 25), in addition to setting the event count for each module along the edge. In this example, runtime is calculated from the MAC frequency, while the counts for

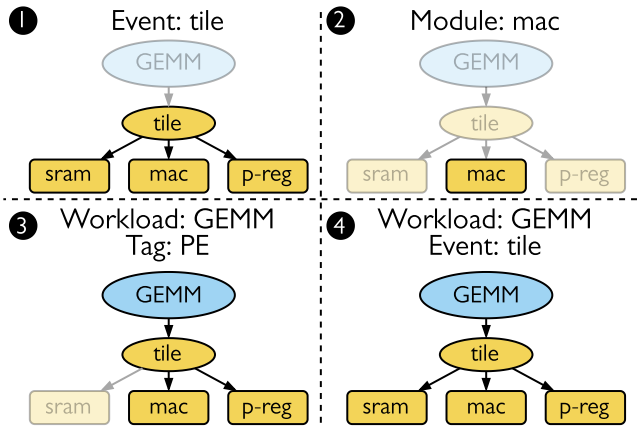

Fig. 7: Scope-based metric retrieval in *Archx*.

the *mac* and *p-reg* modules are determined from the array shape. Next, the *sram* event is computed by scaling the number of reads and writes by the array shape and bitwidth, so the performance graph captures both computation and memory cost for the MA operation.

The resulting graph after applying each performance model is a weighted *A-Graph* that captures the hierarchical interactions between each stack within the described system.

C. Phase 3: Metric Retrieval

Archx empowers designers with detailed metric analysis through its scope-based retrieval and aggregation exposure.

Scope-based Metric Retrieval: In order to fully understand a system's behavior, designers must be able to view resulting metrics at varying levels of granularity within the system. To enable this, *Archx* provides scope-based metric retrieval, allowing designers to select a metric and define the scope of interest for analysis. There are four knobs for such retrieval:

- Event: limits scope to an event and its sub-events.
- Tag: limits scope to events that decompose into modules sharing the same tag.
- Workload: Selects an event that is treated as a root node, aggregating the sub-tree of events and modules under it.
- Module: limits scope to a single module, aggregating all events that decompose into it.

The event and tag knobs are mutually exclusive, but either can combine with the workload knob. Graph traversal always starts from the leaf module nodes, aggregating upward until reaching the root of the specified scope, allowing designers to focus on the scope of interest for hierarchical analysis.

Figure 7 showcases several examples of metric retrieval and scope selection. ❶ With no workload specified, the metrics of a single *tile* event will be retrieved from all modules. ❷ retrieves the metrics of the specific module *mac*, while all others are ignored. ❸-❹ both specify *GEMM* as the workload. ❸ retrieves the metrics based on the tag *pe*. Therefore, *sram* metrics, with no tag of *pe*, are excluded, as detailed in Listing 1 line 24. ❹ With a workload specified, the metrics of all *tile* events will be retrieved, instead of a single *tile* in ❶.

TABLE II: Implemented case studies.

System	Technology	Node	Application	Architecture
Emerging	Superconducting	10 KA/cm2	FIR	Tap array [13]
			CNN	PE array [11]
	CMOS	7nm	TNN	TNN column [45]
Conventional	CMOS	45nm	FFT	Butterfly array
			GEMM	Systolic array
		45nm	GEMM	RISC-V core [79]
		4nm	LLM	NVIDIA H200 [80]

Aggregation Exposure: In addition to scope-based retrieval, *Archx* exposes the aggregation of metrics along each edge in the graph. In *Archx*, as events are aggregated during metric retrieval, the hierarchical aggregation is saved as log files or directly output to the terminal. This allows designers to track how metrics are aggregated for better understanding.

Through scope-based metric retrieval and aggregation exposure, *Archx* enables designers to understand and analyze all design points at desired granularity with high explainability.

V. Experimental Setup

To validate *A-Graph* and *Archx*'s usability, we implement multiple case studies on *Archx*, for both emerging and conventional systems (Table II). We compare simulation fidelity with that of EDA tools for CMOS technologies, and against prior works for non-CMOS technologies.

A. Emerging Systems

Superconducting: Superconductors are metals that exhibit zero electrical resistance at low temperatures, e.g., 10mK∼70mK, promising ultra-low power and ultra-high efficiency [81]. A popular superconducting logic family is single flux quantum (SFQ) logic, which uses the Josephson Junction (JJ) as its fundamental switching element [81], [82]. Superconducting circuits transmit information by quantizing magnetic flux, encoding data as discrete flux quanta (spikes) rather than continuous voltage levels [81], achieving a substantial decrease in power compared to conventional CMOS designs [83], [84] while unlocking higher clock frequencies [82], [85].

We validate superconducting through a finite impulse response (FIR) tap array [13] and a convolutional neural network (CNN) PE array [11]. Both architectures are implemented using the open-source MIT-LL SFQ5ee 10 kA/cm^2 process [86] and simulated with WRSPICE. The resulting metrics are area as the number of Josephson Junctions (JJs), dynamic and leakage power, and throughput.

Neuromorphic: Neuromorphic computing is an emerging AI workload that models the behavior of biological neurons, commonly implemented as either temporal neural networks (TNNs) [87], [88] or spiking neural networks (SNNs) [28]–[31]. TNNs operate on temporal-coded spikes to reduce switching activity, while SNNs rely on rate-coded spikes.

In this study we focus on TNNs, constructing a single TNN column, a layer of a TNN. We validate results against the *full EDA flow* by using Cadence Genus 21.19 for synthesis and Cadence Innovus 19.10 for place-and-route with the

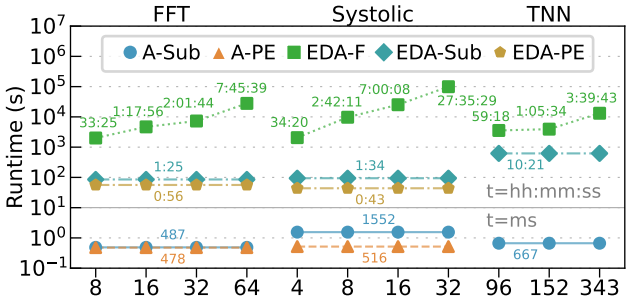

Fig. 8: EDA vs *Archx* (A) runtime across configurations. -PE and -Sub denote running *A-Graph* or EDA flow for the full PE and submodules. -F denotes running full design EDA flow.

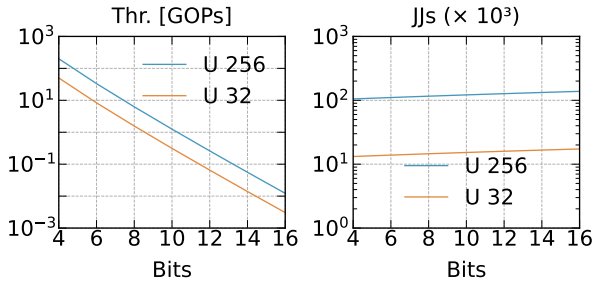

Fig. 9: Validation of FIR bitwidth (x axis).

ASAP7 [89] and customized TNN7 [90] process nodes and PDKs. The hardware is implemented as either standard cells in Verilog, or circuit macros, with floorplan utilization set to 70% for fair comparison. The TNN utilizes a faster 100 MHz clock, and a slower gamma clock of 0.378 MHz.

B. Conventional CMOS Systems

We model conventional CMOS systems, showcasing the flexibility of *A-Graph*. All hardware is implemented similarly to the neuromorphic methodology, using NanGate45 [91] as the process node. The hardware for the fast Fourier transform (FFT), GEMM, and RISC-V is designed at a 400 MHz clock frequency. Additionally, we scale evaluation to the system level by deploying a large language model (LLM), GPT2 [92], on an NVIDIA H200 NVL system to compare against *Archx*.

VI. Case Study

A. Simulation speedup

Figure 8 compares the runtime of the full EDA flow, module database generation, and *Archx* for CMOS. Compared to the full EDA flow, *Archx* is up to $10^5\times$ faster, or $10^3\times$ faster including database generation. Moreover, *Archx* generates all configurations from a single description within milliseconds. With the short generation and execution times, Archx is capable of exploring thousands of design points in the same time it takes EDA tools to retrieve results for a single configuration. *Both demonstrate Archx's programmability, delivering fast DSE compared to baseline results.*

TABLE III: Validation of FIR power results.

Metric	Archx	Baseline	Relative error (%)
Dynamic power (μW)	8.125	8.4	-3.27
Leakage power (mW)	8.4	8.4	1.81×10^{-14}

TABLE IV: Validation of CNN Results. 3D PE array has a dimension of $64 \times 64 \times 32$.

Metric	Archx	Baseline	Relative error (%)
Area w/o overhead (JJ)	13.1M	15.4M	-14.89
Area w/ overhead (JJ)	14.9M	15.4M	-2.97
Power (W)	3.8	3.9	-2.54
Throughput (TMACs)	409.6	409	0.15

B. Emerging Systems

1) Superconducting: This section covers both FIR and CNN case studies.

Finite Impulse Response: An FIR filter computes a finite-duration impulse response using sequential taps, each comprising of a shift register, multiplier, and adder. The multiplier and adder are implemented with a U-SFQ data processing unit (DPU). Leveraging superconducting and unary computing, an FIR can be implemented with improved efficiency at low bit resolutions [13].

Without access to the original throughput and area values, we reproduce their reported trends in Figure 9. Our results yield nearly identical lines for the U-SFQ 32- and 256-tap FIR arrays (U 32, U 256) from [13]. We implement the DPU (U-SFQ multiplier and adder) in *Archx* and model both dynamic and leakage power, as shown in Table III. Our results exhibit low relative error for both. Given that leakage power dominates total power in superconducting designs, the absolute error in total power is also small, yielding nearly identical results to the baseline. *Archx's ability to achieve high fidelity stems from its flexibility, enabling modeling for emerging systems.*

Convolutional Neural Network: CNNs are deep learning models that excel at feature extraction, specifically for image classification. Similar to GEMMs, these networks require highly parallelized hardware for efficient computation. Prior superconducting work has proposed a 3D PE array, with a scale-up array size of $64 \times 64 \times 32$ [11].

We implement this 3D PE array in *Archx* to study a matrix convolution workload, reporting area, power, and throughput, as shown in Table IV. Our results match the baseline across all metrics other than area. In place of JTL chains, common for long connections, the baseline utilizes passive transmission lines (PTLs) as overhead. Including this overhead reduces the error from 15% to 3%. Table V further breaks down resources that are not reported in [11]. *Both of these demonstrate Archx's explainability, exposing the system behavior that drives the understanding and attribution of emerging systems.*

2) Neuromorphic Computing: Figure 10 illustrates *Archx*'s modeling of a TNN column architecture across 3 different input neuron sizes. Both area and leakage power show low

TABLE V: Validation of CNN array breakdown. S denotes splitters. NDRO is for Non-Destructive Readout memory.

Module	MAC	MAC S	Weight S	Input S	NDRO
Area (JJ)	12.3M	393.2K	393.2K	393.2K	448
Power (mW)	3801.08	785.86	785.86	785.86	0.6

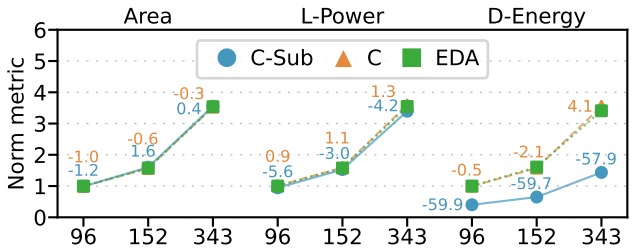

Fig. 10: Validation of TNN input-size (x axis). Each TNN column (C) has 96, 152, or 343 input neurons, with fixed 2 output neurons. All metrics are normalized to the full EDA results of the 96-input column. C-Sub denotes the fine-grained *Archx* implementation using submodules of the column.

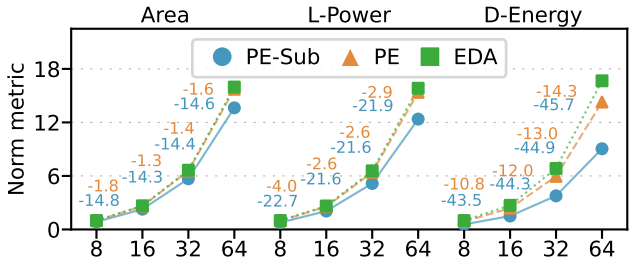

Fig. 11: Validation of FFT array size (x axis). All metrics, area, leakage power (L-Power), and dynamic energy (D-Energy), are normalized to the full EDA results of the 8×3 array. Relative errors (%) are shown above each plot. PE in orange means *Archx* uses the full butterfly unit directly in the database. PE-Sub means *Archx*'s database uses submodules of a PE.

relative error, capped at 1.3% for both C-Sub and C configurations. The discrepancy in C-Sub's dynamic energy arises from submodule EDA macro results not accounting for the actual circuit fanout load. When capturing this load in the column correctly, the gap is closed, as seen in C. *These results underscore the importance of selecting the correct module granularity for high fidelity, while trends still remain present at lower fidelity, demonstrating the flexibility for apples-to-apples comparison.*

C. Conventional CMOS Systems

1) Fast Fourier Transform: We implement a flattened FFT with all butterfly stages fully pipelined. With $2N$ inputs, the FFT array has a shape of $N \times log_2 N$. This flattened FFT allows one-time input mapping without involving memory accesses.

Figure 11 shows fidelity of the FFT when scaling the array size. We evaluate array sizes ranging from 8×3 to 64×6, with all multipliers and adders in bfloat16 [93]. At the PE granularity, the error is consistently small, capped at 15%. In contrast,

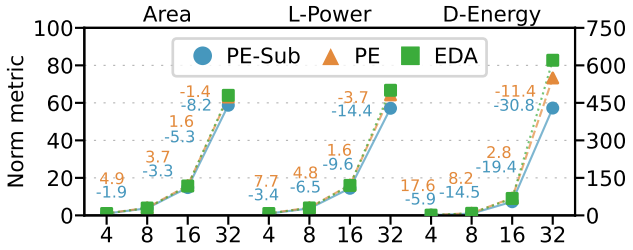

Fig. 12: Validation of systolic array size (x axis). All metrics are normalized to the full EDA results of the 4×4 array. Notations follow these in Figure 11.

TABLE VI: Systolic array configuration and full *Archx* results. Array dimension is the shape of the systolic array. Matrix dimension represents m×k×n of GEMM.

Array dim.	4x4	8x8	16x16	32x32
Matrix dim.	16x16x16	32x32x32	64x64x64	128x128x128
Throughput ($GFLOPs$)	6.4	25.6	102.4	409.6
Compute Area (μm^2)	32.96	127.30	500.09	1982.18
Compute D-Energy (nJ)	16.44	125.00	973.77	7685.43
Compute L-Power (mW)	0.69	2.65	10.38	41.10
Total SRAM (KB)	4	16	64	256
SRAM Area (μm^2)	22.31	77.25	307.13	1133.88
SRAM D-energy (nJ)	0.60	3.73	26.60	191.91
SRAM L-Power (mW)	0.52	1.82	6.87	24.35

the finer PE-Sub granularity shows larger errors when the array size grows. This results from the sub-modules not counting the wiring congestion in the full PE. This under-estimates the resulting metrics, mainly dynamic energy. *Consistent with the neuromorphic case study (Section VI-B2), these results further support choosing the appropriate module granularity for high-fidelity analysis, while lower-fidelity configurations still follow the same trends.*

2) Systolic Array: We implement several weight-stationary systolic arrays, with local buffers, for GEMM workloads. Each PE contains a multiplier, accumulator, registers and extra control logic. Surrounding the PE array are FIFOs for synchronization, an output accumulator for tiled partial sums, and three SRAMs for input, weight, and output buffers.

Table VI reports the array configuration together with the throughput and cost of both the compute array and the SRAM. At 100% utilization, throughput and compute cost scale linearly with array size, since the GEMM workload grows in proportion to the number of PEs. The resulting memory cost is queried from CACTI7 [54].

Figure 12 shows the array scaling of systolic arrays. At smaller array sizes, the PE implementation outperforms the PE-Sub version in fidelity. But at larger array sizes, the PE-Sub implementation outperforms. *Such trade-offs between fidelity and array sizes suggest that users create an ensemble of module databases for different configurations to ensure fidelity, supported through A-Graph's flexibility and programmability. Moreover, Archx showcases programmability by sweeping across 4 total array and workload combinations, while more can be supported.* Though not included here, we use *Archx* to

TABLE VII: Validation of RISC-V GEMM Results.

Metric	Archx	Baseline	Relative error (%)
Area (mm^2)	1.265×10^{-2}	1.271×10^{-2}	-0.510%
Dynamic energy (nJ)	27.522	32.295	-14.78%
Leakage Power (uW)	256.84	255.15	1.01%

TABLE VIII: Validation of GPU GPT-2 Results.

Metric	Archx	Baseline	Relative error (%)
Energy (J)	36.307	36.307	3.057%
Dynamic energy (nJ)	421.756	442.553	0.953%
Leakage power (uW)	86.087	86.087	1.00%

sweep thousands of configurations for accelerating LLMs and quantum error decoding, with costs validated against place-and-route within 15% errors.

3) RISC-V Core: The evaluated RISC-V [79] core is a 5-stage, in-order processor implementing the RV32I ISA. The design includes standard hazard detection and forwarding logic, along with a static not-taken branch predictor. The RV32I ISA does not include multiplication, instead being implemented via bit-serial addition. We map a 2×2 GEMM workload and integrate *Archx* with an external cycle-accurate instruction trace simulator, reporting 3142 cycles (including pipeline stalls). We place-and-route each pipeline stage, extracting their energy from simulated VCD files of circuit switching activity. Each of these stages is then instantiated in *Archx*, with the cycle trace used as the target workload. We intentionally exclude the cache and use this instruction-level model rather than component-level (e.g., McPAT [94]) to demonstrate the flexibility and programmability.

Table VII illustrates the area, dynamic energy, and leakage power of our simulation compared to a full place-and-route implementation. *Similar to other results, Archx proves high fidelity for area and leakage power comparisons, both at or below 1%, while dynamic energy stays reasonably accurate.*

4) GPU System: Executing a workload on a GPU involves host-side orchestration, where the CPU manages data and dispatches kernels. Kernels are hierarchically mapped to grids and thread blocks for parallel execution across streaming multiprocessors. AI workloads further decompose computation into operators, mapping to kernels based on parallelization strategy and hardware utilization.

Table VIII presents results for the GPT-2 workload [92] simulated with a batch size of 32 and sequence length of 2048. The workload is constructed hierarchically, decomposing the application into operators and kernels. We construct the performance and cost model by integrating an in-house GPU kernel micro-benchmarking framework based on prior works [95], [96], leveraging pynvml [97] for metric measurements. *Archx* models the constructed operator–kernel dependency graph, aggregating kernel metrics to operators and the full workload. Baseline metrics are obtained from the same framework as end-to-end measurements, providing a consistent reference for evaluation. *Archx shows high fidelity, producing metrics nearly identical to the baseline. Moreover, Archx retrieves results in seconds, compared to the hours required for full GPU kernel profiling, reinforcing Archx's programmability.*

VII. Discussion

A. Related Work

1) Architecture Modeling: Charm provides a high-level architecture language that expresses relationships between dependent architectural parameters using analytical evaluation [40]. It builds an internal dependency graph to solve these relationships and apply constraints and optimizations for efficient modeling. Both Charm and *A-Graph* use high-level analytical models and constrain DSE. *A-Graph* goes further by decomposing these models into real hardware metrics for cross-stack evaluation, limiting DSE to valid design points.

2) Performance and Cost Modeling: There exist prior works that attempt to project architecture predictions by mapping expert C code to pre-synthesized RTL blocks and memory models [41]. Additional works instead leverage lookup tables to retrieve energy costs from primitive and compound compute and memory components [46]. Finally, recent works extend graph neural networks to HDL compilation, generating a hierarchical graph representation for accurate cost estimation [42]. While all are effective, each restricts itself to a specific target or stack, limiting generality to emerging systems.

3) Agile Chip Generation: Prior efforts have considered chip generation from #pragma annotated C code, mapping execution onto precomputed hardware primitives for both ASIC FPGA and spatial accelerators [43], [44]. Furthermore, recent works have extended generation to neuromorphic computing, targeting TNN models and architecture [98]. These frameworks limit themselves to specific accelerators or architecture and thus are not extendable to emerging systems.

B. Limitations of This Work

A-Graph flattens the computational graph into static events and models their performance by capturing sub-event interactions. Performance models can be either written manually by users for emerging systems (e.g., superconducting, neuromorphic) or generated by profiling tools or compilers for mature systems (e.g., CPUs and GPUs). Expertise is required to design circuit-level databases for emerging systems, which can be reused by architecture and system innovations. With mature system stacks, the workflow can be automated for a specific target or stack, something we leave for future works.

VIII. Conclusion

In the golden age of computer architecture, new systems are rapidly expanding the design space, creating a need for fast exploration. We propose *A-Graph* to unify representations across system stacks and separate performance models and cost modeling, enabling flexible, tightly coupled cost co-simulation. Building on these concepts, *Archx* enhances usability through a front-end interface that automatically generates and constrains design spaces. Moreover, *Archx* provides scope-based metric

retrieval and aggregation exposure, improving programmability and explainability for cross-stack cost modeling. Extensive case studies demonstrate *A-Graph*'s usability in cross-stack system representation while showcasing its applicability to emerging and conventional systems.

REFERENCES

[1] J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," *Commun. ACM*, vol. 62, no. 2, p. 48–60, Jan. 2019. [Online]. Available: https://doi.org/10.1145/3282307

[2] A. Thomas, "Memristor-based neural networks," *Journal of Physics D: Applied Physics*, vol. 46, no. 9, p. 093001, 2013.

[3] A. Chanthbouala, V. Garcia, R. O. Cherifi, K. Bouzehouane, S. Fusil, X. Moya, S. Xavier, H. Yamada, C. Deranlot, N. D. Mathur *et al.*, "A ferroelectric memristor," *Nature materials*, vol. 11, no. 10, pp. 860–864, 2012.

[4] C. Li, M. Hu, Y. Li, H. Jiang, N. Ge, E. Montgomery, J. Zhang, W. Song, N. Dávila, C. E. Graves *et al.*, "Analogue signal and image processing with large memristor crossbars," *Nature electronics*, vol. 1, no. 1, pp. 52–59, 2018.

[5] L. Chua, "Memristor-the missing circuit element," *IEEE Transactions on circuit theory*, vol. 18, no. 5, pp. 507–519, 2003.

[6] P. Yao, H. Wu, B. Gao, J. Tang, Q. Zhang, W. Zhang, J. J. Yang, and H. Qian, "Fully hardware-implemented memristor convolutional neural network," *Nature*, vol. 577, no. 7792, pp. 641–646, 2020.

[7] R. S. Williams, "How we found the missing memristor," *IEEE spectrum*, vol. 45, no. 12, pp. 28–35, 2008.

[8] P. B. Allen and B. Mitrović, "Theory of superconducting tc," *Solid state physics*, vol. 37, pp. 1–92, 1983.

[9] J. Clarke and F. K. Wilhelm, "Superconducting quantum bits," *Nature*, vol. 453, no. 7198, pp. 1031–1042, 2008.

[10] E. Savitskii, *Superconducting materials*. Springer Science & Business Media, 2012.

[11] P. Gonzalez-Guerrero, K. Huch, N. Patra, T. Popovici, and G. Michelogiannakis, "Toward practical superconducting accelerators for machine learning using u-sfq," *J. Emerg. Technol. Comput. Syst.*, vol. 20, no. 2, Jun. 2024. [Online]. Available: https://doi.org/10.1145/3653073

[12] G. Michelogiannakis, A. Butko, P. Gonzalez-Guerrero, D. Vasudevan, M. Gay Bautista-Jurney, C. Grace, P. Zarkos, and J. Shalf, "Unconventional compute methods and future challenges for superconducting digital computing," *Frontiers in Materials*, vol. 12, p. 1618615, 2025.

[13] P. Gonzalez-Guerrero, M. G. Bautista, D. Lyles, and G. Michelogiannakis, "Temporal and sfq pulse-streams encoding for area-efficient superconducting accelerators," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 963–976. [Online]. Available: https://doi.org/10.1145/3503222.3507765

[14] G. Tzimpragos, D. Vasudevan, N. Tsiskaridze, G. Michelogiannakis, A. Madhavan, J. Volk, J. Shalf, and T. Sherwood, "A computational temporal logic for superconducting accelerators," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 435–448. [Online]. Available: https://doi.org/10.1145/3373376.3378517

[15] B. Dong, F. Brückerhoff-Plückelmann, L. Meyer, J. Dijkstra, I. Bente, D. Wendland, A. Varri, S. Aggarwal, N. Farmakidis, M. Wang *et al.*, "Partial coherence enhances parallelized photonic computing," *Nature*, vol. 632, no. 8023, pp. 55–62, 2024.

[16] B. J. Shastri, A. N. Tait, T. Ferreira de Lima, W. H. Pernice, H. Bhaskaran, C. D. Wright, and P. R. Prucnal, "Photonics for artificial intelligence and neuromorphic computing," *Nature Photonics*, vol. 15, no. 2, pp. 102–114, 2021.

[17] B. L. Drake, R. P. Bocker, M. E. Lasher, R. H. Patterson, and W. J. Miceli, "Photonic computing using the modified signed-digit number representation," *Optical Engineering*, vol. 25, no. 1, pp. 38–43, 1986.

[18] P. Antonik, N. Marsal, D. Brunner, and D. Rontani, "Human action recognition with a large-scale brain-inspired photonic computer," *Nature Machine Intelligence*, vol. 1, no. 11, pp. 530–537, 2019.

[19] C. Ríos, N. Youngblood, Z. Cheng, M. Le Gallo, W. H. Pernice, C. D. Wright, A. Sebastian, and H. Bhaskaran, "In-memory computing on a photonic platform," *Science advances*, vol. 5, no. 2, p. eaau5759, 2019.

[20] G. Van der Sande, D. Brunner, and M. C. Soriano, "Advances in photonic reservoir computing," *Nanophotonics*, vol. 6, no. 3, pp. 561–576, 2017.

[21] J. L. O'brien, "Optical quantum computing," *Science*, vol. 318, no. 5856, pp. 1567–1570, 2007.

[22] D. Schrader, I. Dotsenko, M. Khudaverdyan, Y. Miroshnychenko, A. Rauschenbeutel, and D. Meschede, "Neutral atom quantum register," *Physical Review Letters*, vol. 93, no. 15, p. 150501, 2004.

[23] L. Isenhower, E. Urban, X. Zhang, A. Gill, T. Henage, T. A. Johnson, T. Walker, and M. Saffman, "Demonstration of a neutral atom controlled-not quantum gate," *Physical review letters*, vol. 104, no. 1, p. 010503, 2010.

[24] T. Rudolph, "Why i am optimistic about the silicon-photonic route to quantum computing," *APL photonics*, vol. 2, no. 3, 2017.

[25] R. Blatt and C. F. Roos, "Quantum simulations with trapped ions," *Nature Physics*, vol. 8, no. 4, pp. 277–284, 2012.

[26] J. M. Pino, J. M. Dreiling, C. Figgatt, J. P. Gaebler, S. A. Moses, M. Allman, C. Baldwin, M. Foss-Feig, D. Hayes, K. Mayer *et al.*, "Demonstration of the trapped-ion quantum ccd computer architecture," *Nature*, vol. 592, no. 7853, pp. 209–213, 2021.

[27] J. LaRocco, Q. Tahmina, R. Petreaca, J. Simonis, and J. Hill, "Sustainable memristors from shiitake mycelium for high-frequency bioelectronics," *PLoS One*, vol. 20, no. 10, p. e0328965, 2025.

[28] C. Liu, B. Yan, C. Yang, L. Song, Z. Li, B. Liu, Y. Chen, H. Li, Q. Wu, and H. Jiang, "A spiking neuromorphic design with resistive crossbar," in *Proceedings of the 52nd Annual Design Automation Conference*, 2015, pp. 1–6.

[29] Z. He, J. Lin, R. Ewetz, J.-S. Yuan, and D. Fan, "Noise injection adaption: End-to-end reram crossbar non-ideal effect adaption for neural network mapping," in *Proceedings of the 56th Annual Design Automation Conference 2019*, ser. DAC '19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: https://doi.org/10.1145/3316781.3317870

[30] G. C. Adam, B. D. Hoskins, M. Prezioso, F. Merrikh-Bayat, B. Chakrabarti, and D. B. Strukov, "3-d memristor crossbars for analog and neuromorphic computing applications," *IEEE Transactions on Electron Devices*, vol. 64, no. 1, pp. 312–318, 2016.

[31] A. Ankit, A. Sengupta, P. Panda, and K. Roy, "Resparc: A reconfigurable and energy-efficient architecture with memristive crossbars for deep spiking neural networks," in *Proceedings of the 54th Annual Design Automation Conference 2017*, 2017, pp. 1–6.

[32] X. Lin, Y. Rivenson, N. T. Yardimci, M. Veli, Y. Luo, M. Jarrahi, and A. Ozcan, "All-optical machine learning using diffractive deep neural networks," *Science*, vol. 361, no. 6406, pp. 1004–1008, 2018.

[33] S. R. Ahmed, R. Baghdadi, M. Bernadskiy, N. Bowman, R. Braid, J. Carr, C. Chen, P. Ciccarella, M. Cole, J. Cooke *et al.*, "Universal photonic artificial intelligence acceleration," *Nature*, vol. 640, no. 8058, pp. 368–374, 2025.

[34] X. Ding, Y.-P. Guo, M.-C. Xu, R.-Z. Liu, G.-Y. Zou, J.-Y. Zhao, Z.-X. Ge, Q.-H. Zhang, H.-L. Liu, L.-J. Wang *et al.*, "High-efficiency single-photon source above the loss-tolerant threshold for efficient linear optical quantum computing," *Nature Photonics*, pp. 1–5, 2025.

[35] K. Alexander, A. Bahgat, A. Benyamini, D. Black, D. Bonneau, S. Burgos, B. Burridge, G. Campbell, G. Catalano, A. Ceballos *et al.*, "A manufacturable platform for photonic quantum computing," *Nature*, vol. 641, no. 8064, pp. 876–883, 2025.

[36] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko *et al.*, "Highly accurate protein structure prediction with alphafold," *nature*, vol. 596, no. 7873, pp. 583–589, 2021.

[37] O. Hennigh, S. Narasimhan, M. A. Nabian, A. Subramaniam, K. Tangsali, Z. Fang, M. Rietmann, W. Byeon, and S. Choudhry, "Nvidia simnet™: An ai-accelerated multi-physics simulation framework," in *International conference on computational science*. Springer, 2021, pp. 447–461.

[38] L. Cardani, F. Bellini, N. Casali, M. Castellano, I. Colantoni, A. Coppolecchia, C. Cosmelli, A. Cruciani, A. D'Addabbo, S. Di Domizio *et al.*, "New application of superconductors: High sensitivity cryogenic light detectors," *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, vol. 845, pp. 338–341, 2017.

[39] R. Stolz, M. Schmelz, V. Zakosarenko, C. Foley, K. Tanabe, X. Xie, and R. Fagaly, "Superconducting sensors and methods in geophysical applications," *Superconductor Science and Technology*, vol. 34, no. 3, p. 033001, 2021.

[40] W. Cui, Y. Ding, D. Dangwal, A. Holmes, J. McMahan, A. Javadi-Abhari, G. Tzimpragos, F. Chong, and T. Sherwood, "Charm: A language for closed-form high-level architecture modeling," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, 2018, pp. 152–165.

[41] Y. S. Shao, B. Reagen, G.-Y. Wei, and D. Brooks, "Aladdin: A pre-rtl, power-performance accelerator simulator enabling large design space exploration of customized architectures," in *2014 ACM/IEEE 41st International Symposium on Computer Architecture (ISCA)*, 2014, pp. 97–108.

[42] C. Xu, P. Sharma, T. Wang, and L. W. Wills, "Fast, robust, and transferable prediction for hardware logic synthesis," in *2023 56th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023, pp. 167–179.

[43] J. Weng, S. Liu, V. Dadu, Z. Wang, P. Shah, and T. Nowatzki, "Dsagen: Synthesizing programmable spatial accelerators," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 268–281.

[44] S. Liu, J. Weng, D. Kupsh, A. Sohrabizadeh, Z. Wang, L. Guo, J. Liu, M. Zhulin, R. Mani, L. Zhang, J. Cong, and T. Nowatzki, "Overgen: Improving fpga usability through domain-specific overlay generation," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 35–56.

[45] P. Vellaisamy, H. Nair, V. Ratnakaram, D. Gupta, and J. Paul Shen, "Tnngen: Automated design of neuromorphic sensory processing units for time-series clustering," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 5, pp. 2519–2523, 2024.

[46] Y. N. Wu, J. S. Emer, and V. Sze, "Accelergy: An architecture-level energy estimation methodology for accelerator designs," in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2019, pp. 1–8.

[47] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A systematic approach to dnn accelerator evaluation," in *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019, pp. 304–315.

[48] C. Yakopcic, T. M. Taha, G. Subramanyam, R. E. Pino, and S. Rogers, "A memristor device model," *IEEE electron device letters*, vol. 32, no. 10, pp. 1436–1438, 2011.

[49] J. A. Delport, K. Jackman, P. Le Roux, and C. J. Fourie, "Josim—superconductor spice simulator," *IEEE Transactions on Applied Superconductivity*, vol. 29, no. 5, pp. 1–5, 2019.

[50] J. Chan, G. Hendry, A. Biberman, K. Bergman, and L. P. Carloni, "Phoenixsim: A simulator for physical-layer analysis of chip-scale photonic interconnection networks," in *2010 Design, Automation & Test in Europe Conference & Exhibition (DATE 2010)*. IEEE, 2010, pp. 691–696.

[51] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti *et al.*, "The gem5 simulator," *ACM SIGARCH computer architecture news*, vol. 39, no. 2, pp. 1–7, 2011.

[52] A. F. Rodrigues, K. S. Hemmert, B. W. Barrett, C. Kersey, R. Oldfield, M. Weston, R. Risen, J. Cook, P. Rosenfeld, E. Cooper-Balis *et al.*, "The structural simulation toolkit," *ACM SIGMETRICS Performance Evaluation Review*, vol. 38, no. 4, pp. 37–42, 2011.

[53] P. Rosenfeld, E. Cooper-Balis, and B. Jacob, "Dramsim2: A cycle accurate memory system simulator," *IEEE computer architecture letters*, vol. 10, no. 1, pp. 16–19, 2011.

[54] R. Balasubramonian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories," *Transactions on Architecture and Code Optimization*, 2017.

[55] S. Xu, X. Chen, Y. Wang, Y. Han, X. Qian, and X. Li, "Pimsim: A flexible and detailed processing-in-memory simulator," *IEEE Computer Architecture Letters*, vol. 18, no. 1, pp. 6–9, 2018.

[56] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of DNN dataflow: A data-centric approach," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO*. ACM, 2019, pp. 754–768.

[57] Q. Huang, M. Kang, G. Dinh, T. Norell, A. Kalaiah, J. Demmel, J. Wawrzynek, and Y. S. Shao, "Cosa: scheduling by ¡u¿c¡/u¿onstrained ¡u¿o¡/u¿ptimization for ¡u¿s¡/u¿patial ¡u¿a¡/u¿ccelerators," in *Proceedings of the 48th Annual International Symposium on Computer Architecture*, ser. ISCA '21. IEEE Press, 2021, p. 554–566. [Online]. Available: https://doi.org/10.1109/ISCA52012.2021.00050

[58] W. Won, T. Heo, S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "Astra-sim2.0: Modeling hierarchical networks and disaggregated systems for large-model training at scale," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2023, pp. 283–294.

[59] T. Ma, Y. Feng, X. Zhang, and Y. Zhu, "Camj: Enabling system-level energy modeling and architectural exploration for in-sensor visual computing," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–14.

[60] D. Wu, J. Li, R. Yin, H. Hsiao, Y. Kim, and J. S. Miguel, "uGEMM: Unary Computing Architecture for GEMM Applications," in *International Symposium on Computer Architecture*, 2020.

[61] D. Wu and J. S. Miguel, "uSystolic: Byte-Crawling Unary Systolic Array," in *International Symposium on High-Performance Computer Architecture*, 2022.

[62] Z. Pan, J. San Miguel, and D. Wu, "Carat: Unlocking Value-Level Parallelism for Multiplier-Free GEMMs," in *International Conference on Architectural Support for Programming Languages and Operating Systems*, 2024.

[63] D. Price, P. Vellaisamy, J. P. Shen, and D. Wu, "Mugi: Value level parallelism for efficient llms," in *International Conference on Architectural Support for Programming Languages and Operating Systems*, 2026.

[64] P. Date, C. Gunaratne, S. R. Kulkarni, R. Patton, M. Coletti, and T. Potok, "Superneuro: A fast and scalable simulator for neuromorphic computing," in *Proceedings of the 2023 International Conference on Neuromorphic Systems*, ser. ICONS '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3589737.3606000

[65] R. Yin, A. Moitra, A. Bhattacharjee, Y. Kim, and P. Panda, "Sata: Sparsity-aware training accelerator for spiking neural networks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2022.

[66] D. Nikiforov, S. C. Dong, C. L. Zhang, S. Kim, B. Nikolic, and Y. S. Shao, "Rosé: A hardware-software co-simulation infrastructure enabling pre-silicon full-stack robotics soc evaluation," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.

[67] B. Zhao, Z. Li, X. Yu, B. Yuan, C. Zhang, Y. Gao, W. Wang, Q. Mu, S. Wang, H. Sun, T. Yang, M. Zhang, C. Han, P. Xu, and W. Wang, "Eda-q: Electronic design automation for superconducting quantum chip," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2026.

[68] Z. K. Minev, T. G. McConkey, J. Drysdale, P. Shah, D. Wang, M. Facchini, G. Harper, J. Blair, H. Zhang, N. Lanzillo, S. Mukesh, W. Shanks, C. Warren, and J. M. Gambetta, "Qiskit Metal: An Open-Source Framework for Quantum Device Design & Analysis," 2021. [Online]. Available: https://doi.org/10.5281/zenodo.4618153

[69] B. Liu, W. Wen, Y. Chen, X. Li, C.-R. Wu, and T.-Y. Ho, "Eda challenges for memristor-crossbar based neuromorphic computing," in *Proceedings of the 25th Edition on Great Lakes Symposium on VLSI*. Association for Computing Machinery, 2015.

[70] F. Seiler, P. M. Hinkel, A. Jantsch, and N. TaheriNejad, "Atomic: Automatic tool for memristive imply based circuit-level simulation and validation," in *2025 21st International Conference on Synthesis, Modeling, Analysis and Simulation Methods, and Applications to Circuits Design (SMACD)*, 2025.

[71] Y. S. Shao, B. Reagen, G.-Y. Wei, and D. Brooks, "Aladdin: A pre-rtl, power-performance accelerator simulator enabling large design space exploration of customized architectures," in *2014 ACM/IEEE 41st International Symposium on Computer Architecture (ISCA)*, 2014, pp. 97–108.

[72] Z. Fu, A. Manley, and M. Alian, "gem5 co-pilot: Ai assistant agent for architectural design space exploration," 2025. [Online]. Available: https://arxiv.org/abs/2510.19577

[73] Y. Sun, "CRII: OAC: Building Explainable Computer Architecture with Simulation and Visualization Techniques," NSF Award Number 2246035. Directorate for Computer and Information Science and Engineering, Office of Advanced Cyberinfrastructure (OAC). 2023., 2023.

[74] T. Nowatzki, J. Menon, C.-H. Ho, and K. Sankaralingam, "Architectural simulators considered harmful," *IEEE Micro*, 2015.

[75] S. Dave, T. Nowatzki, and A. Shrivastava, "Explainable-dse: An agile and explainable exploration of efficient hw/sw codesigns of deep

learning accelerators using bottleneck analysis," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, 2024.

[76] T. A. Grossman, "Spreadsheet engineering: A research framework," *arXiv preprint arXiv:0711.0538*, 2007.

[77] C. Timsit and S. Zertal, "Using spreadsheets to teach computer architecture." in *CSEDU (2)*, 2010, pp. 101–105.

[78] A. G. Yoder and D. L. Cohn, "Architectural issues in spreadsheet languages," in *Programming Languages and System Architectures: International Conference Zurich, Switzerland, March 2–4, 1994 Proceedings*. Springer, 2005, pp. 245–258.

[79] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanovic, "The risc-v instruction set manual, volume i: User-level isa, version 2.1," p. 133 p, May 2016. [Online]. Available: http://digicoll.lib.berkeley.edu/record/135173

[80] NVIDIA. (2023) NVIDIA H200 Tensor Core GPU: The World's Most Powerful GPU for AI. NVIDIA Corporation. [Online]. Available: https://www.nvidia.com/en-us/data-center/h200/

[81] S. S. Tannu, P. Das, M. L. Lewis, R. Krick, D. M. Carmean, and M. K. Qureshi, "A case for superconducting accelerators," in *Proceedings of the 16th ACM International Conference on Computing Frontiers*, ser. CF '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 67–75. [Online]. Available: https://doi.org/10.1145/3310273.3321561

[82] K. Likharev and V. Semenov, "Rsfq logic/memory family: a new josephson-junction technology for sub-terahertz-clock-frequency digital systems," *IEEE Transactions on Applied Superconductivity*, vol. 1, no. 1, pp. 3–28, 1991.

[83] O. A. Mukhanov, "Energy-efficient single flux quantum technology," *IEEE Transactions on Applied Superconductivity*, vol. 21, no. 3, pp. 760–769, 2011.

[84] Q. P. Herr, A. Y. Herr, O. T. Oberg, and A. G. Ioannidis, "Ultra-low-power superconductor logic," *Journal of Applied Physics*, vol. 109, no. 10, May 2011. [Online]. Available: http://dx.doi.org/10.1063/1.3585849

[85] W. Chen, A. Rylyakov, V. Patel, J. Lukens, and K. Likharev, "Rapid single flux quantum t-flip flop operating up to 770 ghz," *IEEE Transactions on Applied Superconductivity*, vol. 9, no. 2, pp. 3212–3215, 1999.

[86] A. F. Kirichenko, S. Sarwana, and D. Gupta, "Advanced SFQ5ee process at MIT Lincoln Laboratory," in *Proceedings of the IEEE International Superconductive Electronics Conference (ISEC)*, 2003, pp. K4–1.

[87] J. E. Smith, "Space-time algebra: a model for neocortical computation," in *Proceedings of the 45th Annual International Symposium on Computer Architecture*, ser. ISCA '18. IEEE Press, 2018, p. 289–300. [Online]. Available: https://doi.org/10.1109/ISCA.2018.00033

[88] ——, "A temporal neural network architecture for online learning," 2021. [Online]. Available: https://arxiv.org/abs/2011.13844

[89] S. Naffziger, S.-C. Yang, D. J. Carlberg, B. Sarkar, S. K. Sinha, G. Yeric, and M. Haycock, "ASAP7: A 7-nm finfet predictive process design kit," in *Proceedings of the 54th Annual Design Automation Conference (DAC)*. ACM, 2017, pp. 1–6.

[90] H. Nair, P. Vellaisamy, S. Bhasuthkar, and J. P. Shen, "Tnn7: A custom macro suite for implementing highly optimized designs of neuromorphic tnns," in *2022 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, 2022, pp. 152–157.

[91] N. Inc., "Nangate Open Cell Library: 45nm open cell standard cell library," 2009, available at https://www.nangate.com.

[92] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[93] N. Burgess, J. Milanovic, N. Stephens, K. Monachopoulos, and D. Mansell, "Bfloat16 processing for neural networks," *IEEE Xplore*, 2019.

[94] S. Li, J. H. Ahn, R. D. Strong, J. B. Brockman, D. M. Tullsen, and N. P. Jouppi, "Mcpat: an integrated power, area, and timing modeling framework for multicore and manycore architectures," in *Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO 42. New York, NY, USA: Association for Computing Machinery, 2009, p. 469–480. [Online]. Available: https://doi.org/10.1145/1669112.1669172

[95] P. Vellaisamy, T. Labonte, S. Chakraborty, M. Turner, S. Sury, and J. P. Shen, "Characterizing and optimizing llm inference workloads on cpu-gpu coupled architectures," in *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2025, pp. 49–61.

[96] P. Vellaisamy, S. Tripathi, V. Natarajan, S. S. Thenarasu, S. Blanton, and J. P. Shen, "Taxbreak: Unmasking the hidden costs of llm inference through overhead decomposition," *arXiv preprint arXiv:2603.12465*, 2026.

[97] GoAI, "pynvml," https://github.com/gpuopenanalytics/pynvml, 2025.

[98] P. Vellaisamy, H. Nair, V. Ratnakaram, D. Gupta, and J. P. Shen, "Tnngen: Automated design of neuromorphic sensory processing units for time-series clustering," *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2024.